

A Bounded Reproducibility Capsule for Public-Data Benchmarking and Preserved Negative Gates

Robert Ashworth LumenCore Preprint draft v0.1 - 2026-07-21 Not peer reviewed. No external validation or CODECHECK certificate has been issued.

Abstract

Computational claims are difficult to evaluate when source code, inputs, dependency versions, expected outputs, and failed decision gates are separated across a large project. This paper defines a narrow, public, executable capsule that packages two frozen benchmarks derived from U.S. Energy Information Administration Form EIA-930 data and one deterministic synthetic falsification suite. The capsule declares six output files and 31 bounded assertions. It also records exact dependency closure, source hashes, environment controls, a scoped software bill of materials, and negative promotion outcomes.

An author-operated rebuild from a deterministic source ZIP in a digest-pinned Ubuntu 24.04 x86-64 container with CPython 3.11.9 reproduced all three suites, all 31 assertions, and all 28 selected fixture tests with execution networking disabled. This result establishes first-party executable reproducibility only. It does not establish independent reproduction, scientific validity, field performance, agency approval, realized savings, patent scope, profitable trading, or company valuation. The capsule is prepared for independent execution through the CODECHECK community workflow, but no external request, assignment, execution, or certificate is claimed in this draft.

1. Research Question And Claim Boundary

The single computational question is:

Can an independent executor use the named public source files, frozen input, dependency lock, and deterministic controls to recreate every file in `codecheck.yml` and reconcile 31 declared assertions across three bounded suites?

The target is executable-computation checking. It is not a test of universal model superiority or operational readiness. A successful independent execution would show that the declared computations can be recreated under the recorded environment. It would not show that the live prospective system satisfies sample gates, that a model saves money in deployment, that a government agency endorses the work, or that the company has any particular valuation.

2. Materials

2.1 Frozen measured-data panel

The measured-data input is `evidence/reproducibility/eia_grid_validation_panel_20260713.json.gz`. It contains 14,704 frozen rows derived from public U.S. Energy Information Administration Form EIA-930 demand and day-ahead demand-forecast data. The repository license records the EIA source acknowledgment, reuse policy, and API terms. No EIA endorsement is implied.

2.2 Deterministic falsification fixtures

The third suite uses 128 deterministic synthetic fixtures for open-set control mapping, including a 36-fixture holdout. These fixtures are intentionally synthetic. They test code behavior and negative-gate preservation, not field performance.

2.3 Frozen protocol

`config/reviewer_reproducibility_protocol_v1.json` is the authority for suite identifiers, expected facts, tolerances, exclusions, environment controls, and claim boundaries. The capsule includes two measured-data suites and one deterministic synthetic suite:

```
1 eia_wave_frozen_holdout
2 eia_residual_frozen_holdout
3 mda_open_set_v2
```

3. Methods

3.1 Environment control

The authoritative environment is Ubuntu 24.04 x86-64 with CPython 3.11.9 and glibc 2.39. The public container recipe pins the Ubuntu platform manifest, uv archive hash and version, Python version, and binary-only dependency lock. Execution uses no network, a read-only root filesystem, dropped Linux capabilities, no-new-privileges, bounded memory, CPU, and process limits, and separate read-only input and writable output mounts. A fail-closed verifier reads the OS release, architecture, Python and libc versions, deterministic environment variables, dependency-lock hash, release-manifest hashes, and source provenance before issuing a bounded receipt.

3.2 Execution

From the repository root, the declared workflow is:

```
python code/ops/VERIFY_CODECHECK_REVIEWER_RUNTIME.py --check-only
python code/ops/VERIFY_REVIEWER_DEPENDENCY_LOCK.py
python -m pip install --disable-pip-version-check --require-hashes --only-binary=:all: --requirement
requirements-reviewer-ubuntu-py311.lock
python -m pip check
python code/ops/RUN_REVIEWER_REPRODUCIBILITY_CAPSULE.py --with-fixture-tests --run-dir out/codecheck_eia
```

The run must recreate these six manifest outputs:

- `out/codecheck_eia/reviewer_reproducibility_receipt.json`
- `out/codecheck_eia/reviewer_suite_sbom.cdx.json`
- `out/codecheck_eia/logs/eia_wave.log`
- `out/codecheck_eia/logs/eia_residual.log`
- `out/codecheck_eia/logs/mda_open_set.log`
- `out/codecheck_eia/logs/fixture_tests.log`

3.3 Assertion policy

The runner compares observed facts with the frozen protocol. Identity, structural, coverage, and gate assertions remain exact unless the protocol explicitly records a tolerance. Every failed promotion or coverage gate is preserved in the receipt rather than hidden by aggregate performance.

4. Frozen Results To Reproduce

4.1 EIA wave holdout

- Panel rows: 14,704
- Holdout rows: 1,525 across eight balancing authorities
- Selected candidate: `lissajous_phase_paths`
- Best declared strategy: `autoregressive_ridge_p14`
- Best MASE: 0.47945937271811834
- Kuramoto MASE: 1.253508683225091
- Promotion gate: failed
- Field validation complete: false

The geometry candidate did not win this bounded comparison. That negative result is part of the scientific record.

4.2 EIA residual holdout

- Panel rows: 14,704
- Holdout rows: 1,176 across eight balancing authorities
- Selected candidate and best declared strategy: `xgboost_residual`
- Best MASE reference: 0.21211186326437864
- Declared baseline comparisons: six
- Promotion gate: failed
- Coverage gate: failed
- Field validation complete: false

The aggregate metric is favorable relative to the declared baselines, while the complete protocol still refuses promotion. Both facts must be reported together.

4.3 MDA open-set v2

- Deterministic fixtures: 128
- Holdout fixtures: 36
- Candidate micro-F1: 0.9433962264150945
- Supported coverage: 0.9583333333333334
- Unsupported mapping rate: 0.0
- Promotion gate: failed
- Operational or field claim allowed: false

This suite is a deterministic software falsification test, not measured operational evidence.

5. Preserved Failure And Protocol Amendment

An early clean Ubuntu replay exposed a missing ignored input path. A later replay passed 30 of 31 assertions but measured a 0.4269440625 percent XGBoost CPU histogram MASE difference relative to the Windows reference. Selected-model identity, comparison count, coverage state, and every decision gate were unchanged.

After observing that difference, the protocol was amended to permit at most one percent relative drift for that XGBoost residual MASE assertion only. This is explicitly a post-observation portability amendment, not a preregistered scientific threshold. The protocol retains the failed GitHub run identifiers and the amendment boundary.

6. Author-Operated Reproducibility Evidence

The current public author receipts record a no-cache rebuild of deterministic source bundle 0fa73c04454cfbd94f819b9584ccd2167982e3a95542e9345f687a1c823463ab from source commit b2ede0583776de47459fac4413f2570e2636f199 under the authoritative Linux and Python environment:

- Relevant source clean: true
- Exact dependency closure: true
- Fixture tests passed: 28 of 28 selected tests
- Suites passed: 3 of 3
- Assertions passed: 31 of 31
- External validation complete: false
- Container receipt SHA-256: 560cc3b5f187c0a4add90c527e967de36e13788c53e3bd8aaa63f97c10a3787e
- Capsule receipt SHA-256: f8280773d1fa3e1209f57c64d3fcdf9608e4d633c6a0148b2578fc45728b8a1a
- Capsule payload SHA-256: 92e48edf885e731ff502fa81698052d23d81827c84720231d403343799aff30b
- Source-chain SHA-256: 2b5e73121212fe57b6cb0cd47565419ed88090be609b392c4aed5c35e0e7e3b6

This is first-party evidence. An independent codechecker must execute the reviewed source and document what was checked, how it was checked, and what was reproduced before any external claim changes.

The companion runtime receipt binds the environment claim to 10/10 measured checks: Ubuntu 24.04, Linux, x86-64, CPython 3.11.9, glibc 2.39, five deterministic environment values, and the exact dependency-lock hash. The container receipt cross-locks that runtime receipt, the capsule receipt, release manifest, source bundle, recipe hashes, and isolation controls. All are operator-controlled and leave independent execution and external validation false.

7. Limitations

- 1 The measured-data suites replay a dated frozen panel; they do not evaluate the current live prospective router.
- 2 The capsule checks executable facts, not causal effects, deployment economics, safety, reliability, or generalization beyond the declared data and protocol.
- 3 One suite uses synthetic fixtures and therefore cannot support a field claim.
- 4 The XGBoost portability tolerance was added after a cross-platform failure and is disclosed as post-observation.
- 5 The author-operated replay is not statistically or institutionally independent.
- 6 No CODECHECK issue, codechecker assignment, certificate, journal review, or DOI exists at the time of this draft.

8. Data, Code, And Reproduction Availability

The public source repository is:

<https://github.com/robertashworth1986-debug/lumen-core-public>

The root `README.md`, `codecheck.yml`, and `LICENSE` provide execution, manifest, code-license, and third-party-data terms. The frozen input is committed under `evidence/reproducibility`. The exact dependency lock and runner are committed at the repository root and under `code/ops`.

A stable preprint identifier, immutable source release identifier, CODECHECK register issue, independent execution receipt, and certificate remain open gates.

9. Ethics, Funding, And Competing Interests

This work uses public energy-system data and deterministic synthetic fixtures; it does not contain human-subject data. No external funder is claimed for this capsule. The author is the founder of LumenCore and has an interest in the project. No government, EIA, CODECHECK, university, or commercial endorsement is claimed.

10. Author Contribution

Robert Ashworth conceived the project direction and is responsible for the submitted author materials. AI-assisted software engineering contributed to implementation, testing, documentation, and evidence reconciliation under human direction. The author remains responsible for the accuracy of the manuscript and for all external submissions and attestations.

References

- 1 CODECHECK. Community workflow guide for authors. <https://codecheck.org.uk/guide/community-workflow-author>
- 2 CODECHECK. Configuration file specification 1.0. <https://codecheck.org.uk/spec/config/1.0/>
- 3 U.S. Energy Information Administration. Form EIA-930 data and API. <https://www.eia.gov/opendata/>
- 4 U.S. Energy Information Administration. Copyrights and reuse. https://www.eia.gov/about/copyrights_reuse.php
- 5 U.S. Energy Information Administration. Open Data terms of service. <https://www.eia.gov/opendata/terms-of-service.php>

Authoritative Markdown source SHA-256: 2cc9a001b3a4c6a830b04b626f11b61f3a06ad49bf56e8599d4bb35c15b9f1a6